

Session 03 – Generative Modelling: Autoencoders and GANs

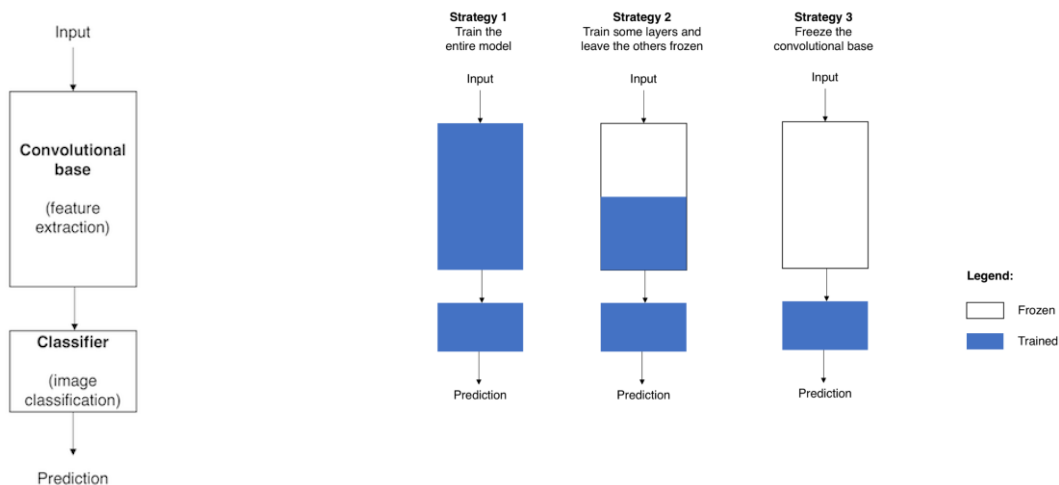
Dr Ivan Olier

I.Olier@ljmu.ac.uk

1

Pre-training

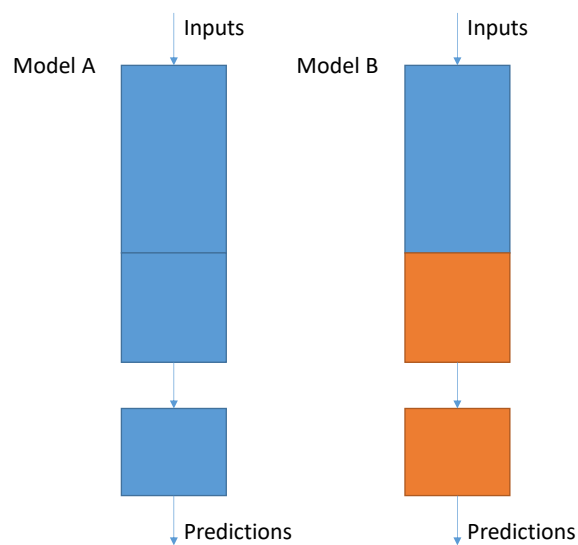
Freezing layers



3

Re-using a model – Transfer learning

- We can train a model from scratch, as we have done so far.
- Also, we can re-use a trained model that was used for a similar task.
- Part of the trained model network is removed and new set of layers are added.
- During training, the pre-trained layers are frozen, so the new layers are the only ones that will adapt.
- In ML, this is known as Transfer Learning, which is the ability to transfer the knowledge previously learnt to a new model so it doesn't need to be trained from scratch.
- This is extremely powerful, and allows us the use of very complex models whilst controlling for overfitting.



4

ImageNet

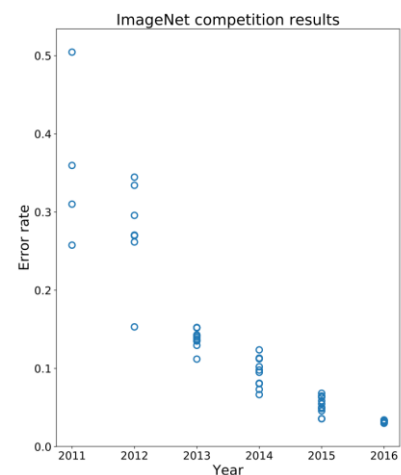
- ImageNet is a large-scale visual recognition challenge designed for training and benchmarking computer vision models.
- It consists of over **14 million images** with more than **20,000** object categories, making it one of the **largest** and most diverse image datasets available.
- ImageNet has been used to pre-train deep neural networks, such as AlexNet, VGGNet, and ResNet, that have achieved state-of-the-art results on a variety of computer vision tasks.
- ImageNet continues to be a popular dataset for research in computer vision and has contributed significantly to the advancement of the field.



Pre-trained models on ImageNet

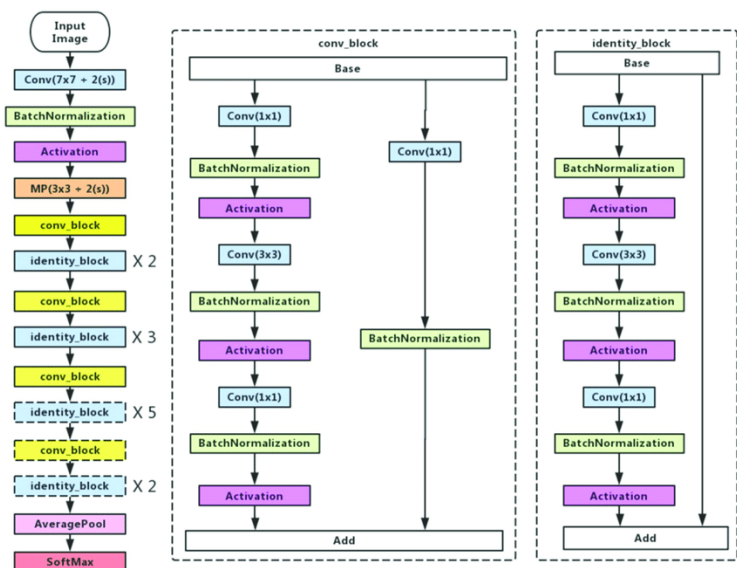
- ImageNet is a database of more than 14'M images and more than 20,000 categories.
- They run a challenge every year.
- CNN using pre-trained models has been consistently winning.
- Currently there are several pre-trained models already available that can be used for more specific tasks:
 - Xception
 - VGG16
 - VGG19
 - ResNet50
 - InceptionV3
 - InceptionResNetV2
 - MobileNet
 - MobileNetV2
 - DenseNet
 - NASNet

- This has actually revolutionised the work with images.



ResNet50

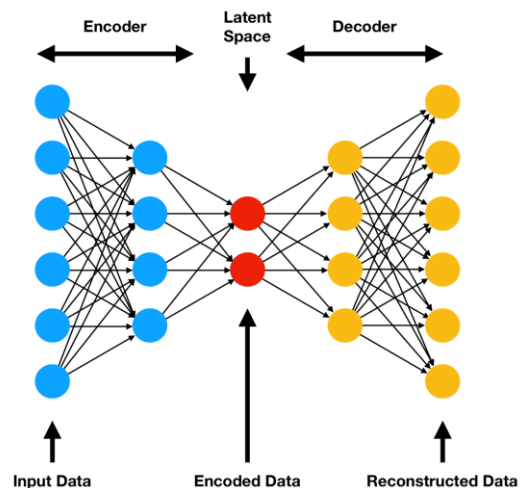
- ResNet50 is a variant of the Residual Network (ResNet) architecture for image classification tasks.
- It consists of 50 convolutional layers and residual connections that allow information to be directly passed between layers, improving the accuracy of the model.
- ResNet50 has been pre-trained on large image datasets and is often used for transfer learning on other image classification tasks.
- ResNet50 is a popular choice for image classification due to its ability to train very deep neural networks with hundreds of layers.



Autoencoders and deep learning

Autoencoders

- Autoencoders are neural networks that can be used to reduce the data into a low dimensional latent space by stacking multiple non-linear transformations(layers).
- They have a encoder-decoder architecture.
- Encoder – maps the input to a (usually low-dimensional) latent space.
- Decoder – reconstructs the input from the latent space.
- They are trained using back propagation for accurate reconstruction of the input.
- If the latent space has lower dimensions than the input, autoencoders can be used for dimensionality reduction.
- By intuition, these low dimensional latent variables should encode most important features of the input since they are capable of reconstructing it.

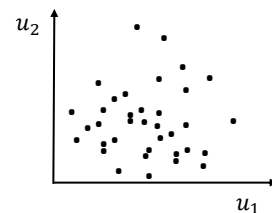
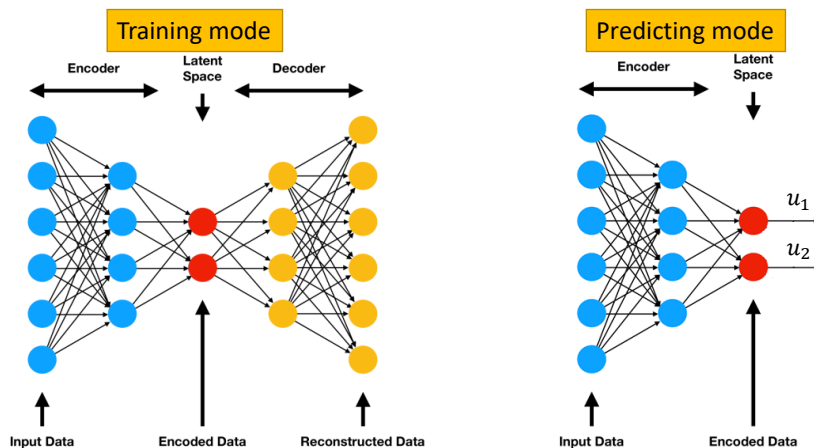


AI - Prof Ivan Olier

9

9

Autoencoders – dimensionality reduction



Low-dimensional data representation. If U is 2-D, the data can be visualised (e.g. producing a scatter plot)

- In training mode, reconstruction error is minimised by error backpropagation optimisation algorithms
- In predicting mode, only the encoder part is used. The output is the encoded data which lay on a low-dimensional space

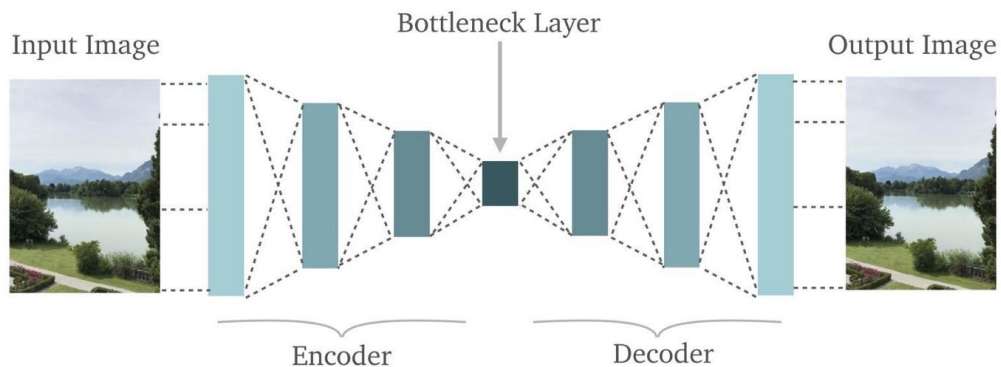
AI - Prof Ivan Olier

10

10

Autoencoders and deep learning

- Autoencoders can also be built using other neural networks architectures as CNNs and LSTMs.
- In image analysis, CNN autoencoders are used for automatically-generated feature extraction.
- Extracted features can then be used as inputs to any ML model.



Upsampling

- Upsampling means increase the dimensions of an image.
- It is used in decoders of segmentation algorithms, creating an image of a random vector sample with GANs and improving an image's quality, etc.
- The upsampling technique increases the resolution as well as the size of the image.
- There are two ways of upsampling.
 - Either we scale the original image up, or
 - we use something called Transpose convolution/deconvolution.

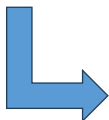


Scaling

- The new pixels that are added are generated with some kind of interpolation/Scaling method.
- Methods:
 - **Nearest neighbour interpolation:** in which the upsampled pixel is a copy of the closest pixel from the pool.
 - **Bilinear interpolation:** we calculate the weighted distance between each nearest pixel using linear interpolation and calculate the new pixel.
 - **Bicubic interpolation:** we choose the nearest pixel in this approach as well, but instead of linear interpolation to calculate the new pixel value, we have to choose 3rd order polynomial interpolation.

Scaling

1	4	8
8	3	9
2	5	4



Nearest Neighbour

1	1	1	4	4	4	8	8	8
1	1	1	4	4	4	8	8	8
1	1	1	4	4	4	8	8	8
8	8	8	3	3	3	9	9	9
8	8	8	3	3	3	9	9	9
8	8	8	3	3	3	9	9	9
2	2	2	5	5	5	4	4	4
2	2	2	5	5	5	4	4	4
2	2	2	5	5	5	4	4	4

Bilinear

1	1	2	3	4	5	7	8	8
1	1	2	3	4	5	7	8	8
3	3	3	4	4	5	7	8	8
6	6	5	4	3	5	7	9	9
8	8	6	5	3	5	7	9	9
6	6	5	4	4	5	6	7	7
4	4	4	4	4	5	5	6	6
2	2	3	4	5	5	4	4	4
2	2	3	4	5	5	4	4	4

Transposed Convolution

- Used to increase spatial resolution of feature maps (e.g. in decoders, GANs, UNets).
- Performs the reverse operation of a standard convolution, but not a true inverse.
- Achieved by interleaving input values with zeros, then applying a standard convolution.
- Enables learnable upsampling via trainable weights.
- Common in tasks like image generation, segmentation, and super-resolution.

Input: $\begin{bmatrix} a & b \end{bmatrix}$
 $\downarrow \quad \downarrow$
 Zero-insert: $\begin{bmatrix} a & 0 & b \end{bmatrix}$
 Kernel: $\begin{bmatrix} w1 & w2 \end{bmatrix}$
 Convolve $\rightarrow \begin{bmatrix} a \cdot w2, a \cdot w1 + b \cdot w2, b \cdot w1 \end{bmatrix}$

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

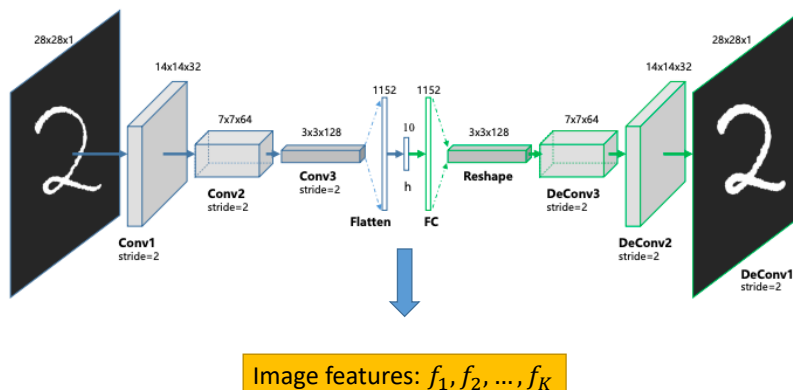
$$\mathbf{K} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\uparrow_2 \mathbf{X} = \begin{bmatrix} 1 & 0 & 2 & 0 & 3 \\ 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 5 & 0 & 6 \\ 0 & 0 & 0 & 0 & 0 \\ 7 & 0 & 8 & 0 & 9 \end{bmatrix} \rightarrow \mathbf{Y} = \begin{bmatrix} 1 & 2 & 2 & 3 \\ 5 & 10 & 10 & 6 \\ 4 & 8 & 8 & 9 \\ 7 & 14 & 14 & 9 \end{bmatrix}$$

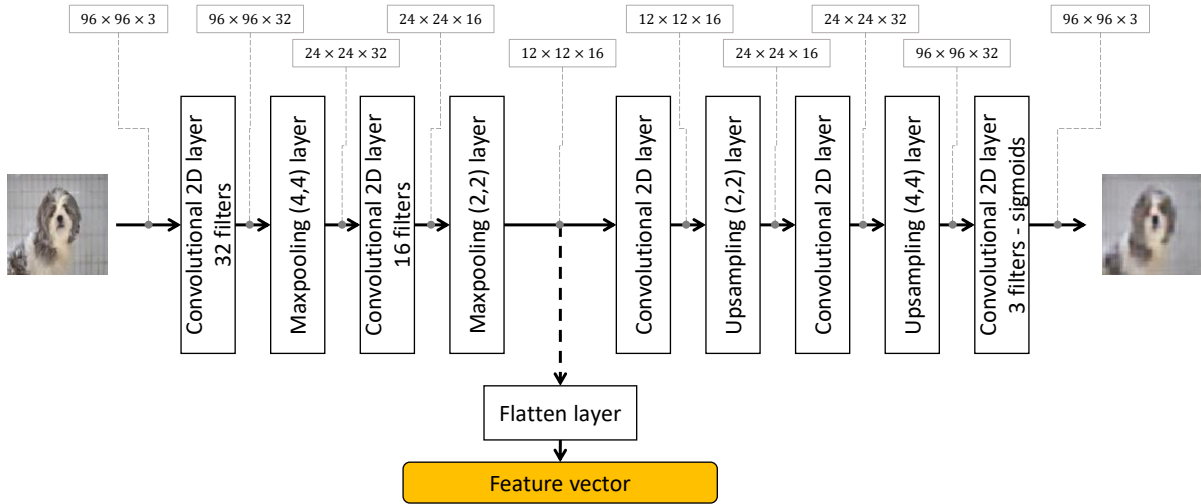
- We use the “stride” parameter to upsampling:
 - Insert $s - 1$ zeros between elements.

CNN autoencoders

- Autoencoder can also be implemented with convolutional neural networks.
- One typical application is to remove image noise,
- But, autoencoders are also used for feature extraction.



Autoencoders in Keras



AI - Prof Ivan Olier

17

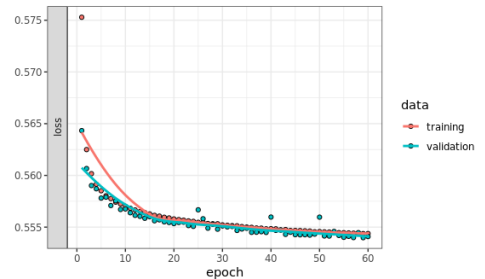
17

Autoencoders in Keras

Model: "model"

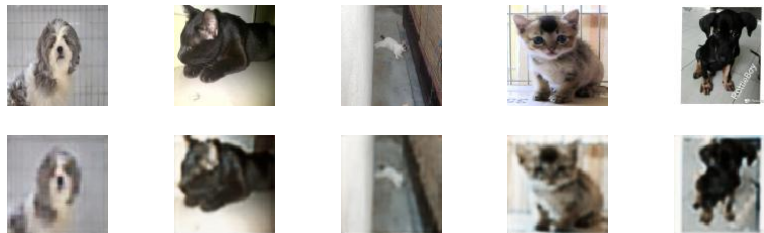
Model architecture in Keras

Layer (type)	Output Shape	Param #
conv2d_input (InputLayer)	(None, 96, 96, 3)	0
conv2d (Conv2D)	(None, 96, 96, 32)	896
max_pooling2d (MaxPooling2D)	(None, 24, 24, 32)	0
conv2d_1 (Conv2D)	(None, 24, 24, 16)	4624
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 16)	0
conv2d_2 (Conv2D)	(None, 12, 12, 16)	2328
up_sampling2d (UpSampling2D)	(None, 24, 24, 16)	0
conv2d_3 (Conv2D)	(None, 24, 24, 32)	4648
up_sampling2d_1 (UpSampling2D)	(None, 96, 96, 32)	0
conv2d_4 (Conv2D)	(None, 96, 96, 3)	867
Total params: 13,347		
Trainable params: 13,347		
Non-trainable params: 0		



Training error curve – It can be seen that model overfitting is well controlled

Input test
images



Reconstructed
images

AI - Prof Ivan Olier

18

18

Generative Modelling

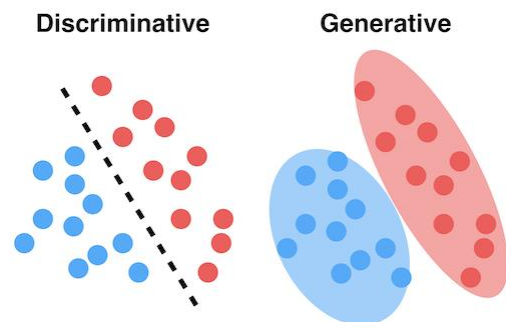
Discriminative vs Generative models

Discriminative models –

- Attempt to find a boundary that separates the data.
- They model the conditional distribution of the classes given the data, $P(Y|X)$
- Most of the well-known ML algorithms are discriminative.

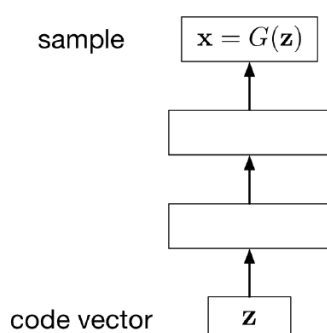
Generative models –

- Attempt to model how the data is placed throughout the space.
- They model the joint distribution between classes and data, $P(X, Y)$.



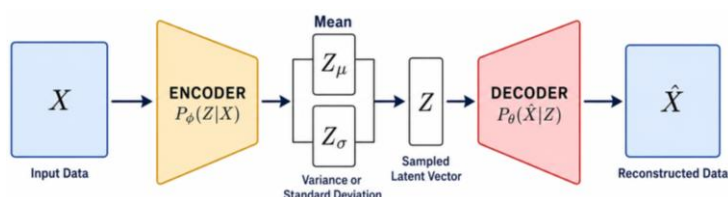
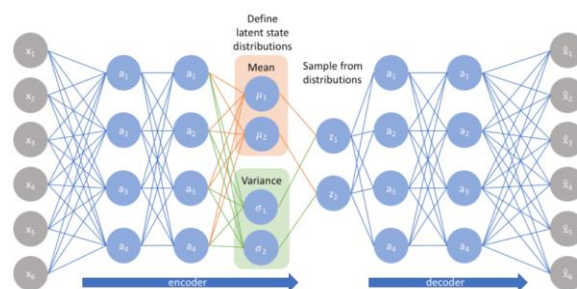
Implicit Generative Models

- **Implicit generative models** implicitly define a probability distribution
- Start by sampling the **code vector $\mathbf{z}$** from a fixed, simple distribution (e.g. spherical Gaussian)
- The **generator network** computes a differentiable function G mapping $\mathbf{z}$ to an $\mathbf{x}$ in data space



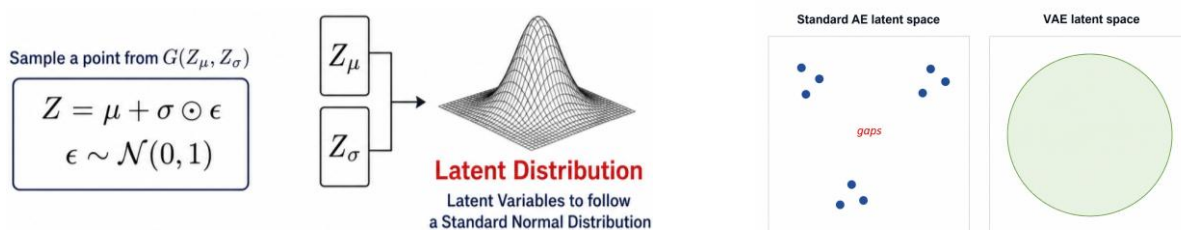
Variational Autoencoders (VAEs)

- VAEs: from reconstruction to generation.
- A standard autoencoder maps an input $\mathbf{x}$ to a single latent vector $\mathbf{z}$, then reconstructs $\hat{\mathbf{x}}$.
- A VAE maps $\mathbf{x}$ to a distribution in latent space: mean $\mu(\mathbf{x})$ and variance $\sigma^2(\mathbf{x})$.
- We sample $\mathbf{z}$ from that distribution and decode it to reconstruct $\mathbf{x}$.
- Because the latent space is probabilistic and regularised, we can sample new $\mathbf{z}$ values to generate new data.



VAEs as data generators

- A normal autoencoder may learn useful codes, but the latent space can contain gaps.
- Randomly sampling from those gaps can produce unrealistic outputs.
- A VAE constrains latent codes to stay close to a simple prior, usually $\mathcal{N}(\mathbf{0}, \mathbf{I})$.
- Generation pipeline:
 - sample $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \rightarrow$ decode $z \rightarrow$ synthetic example.



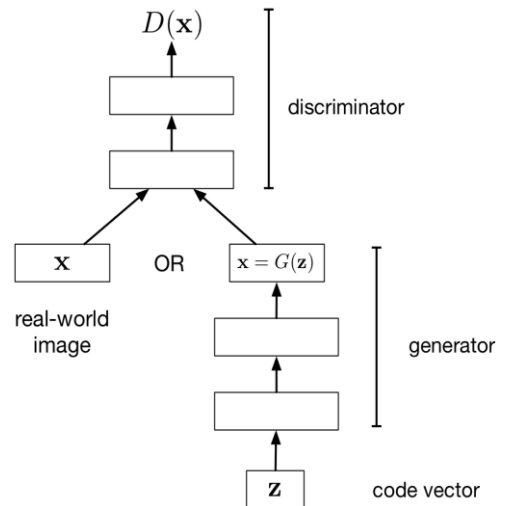
Training a VAE

- Training is based on minimising the reconstruction loss plus the latent-space regularisation.
- Reconstruction loss as traditional AEs:
 - Keeps $\hat{X}$ close to X .
 - Uses MSE or cross-entropy or similar.
 - Preserves data-specific detail.
- Latent-space regularisation:
 - The encoder outputs a distribution $q(z|x) = \mathcal{N}(\mu(x), \sigma(x))$
 - Pushes $q(z|x)$ towards $p(z) = \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - That is, minimises Kullback-Leibler divergence: $D_{KL}(q(z|x)||p(z))$
 - Because of this, the latent space becomes continuous and sampleable:

$$z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

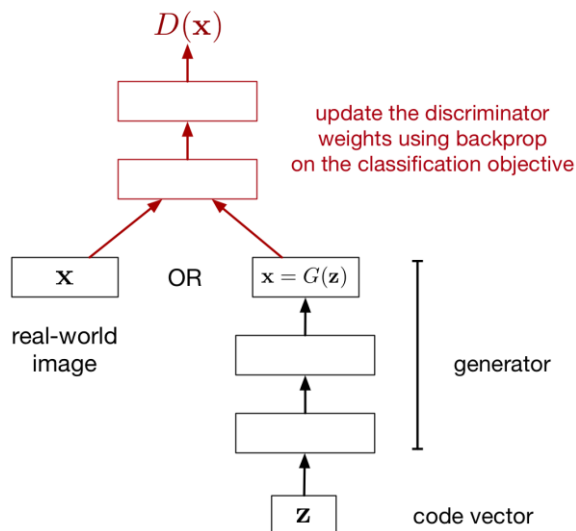
Generative Adversarial Networks

- The advantage of implicit generative models: if you have some criterion for evaluating the quality of samples, then you can compute its gradient with respect to the network parameters, and update the network's parameters to make the sample a little better.
- The idea behind **Generative Adversarial Networks (GANs)**: train two different networks:
 - The **generator network** tries to produce realistic-looking samples
 - The **discriminator network** tries to figure out whether an image came from the training set or the generator network
- The generator network tries to fool the discriminator network

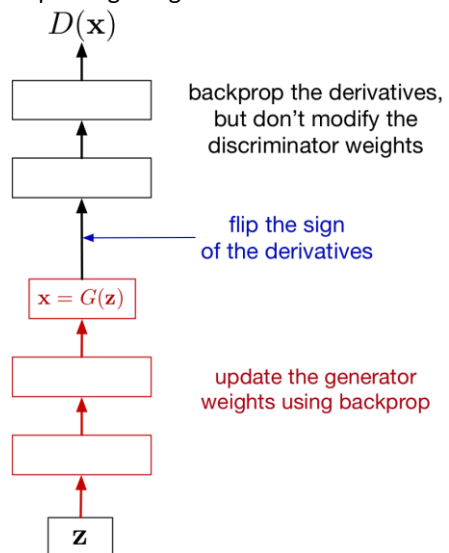


GANs – learning process

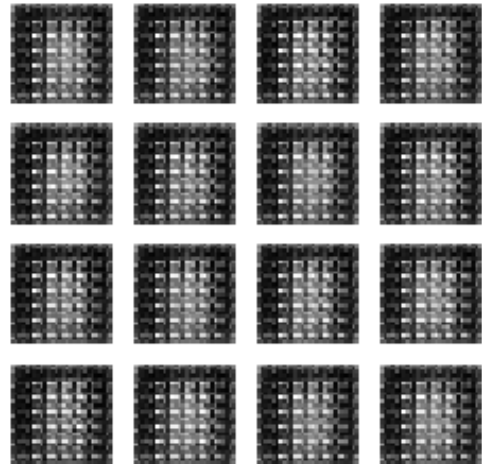
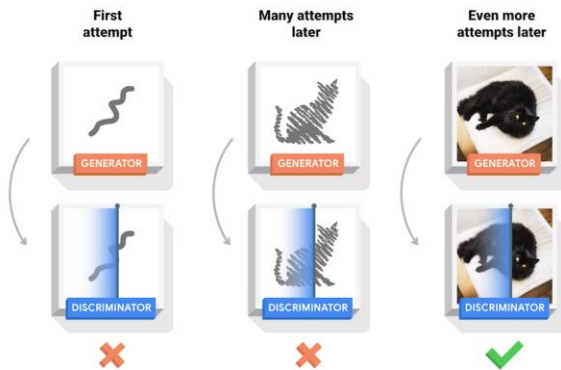
Updating the discriminator



Updating the generator

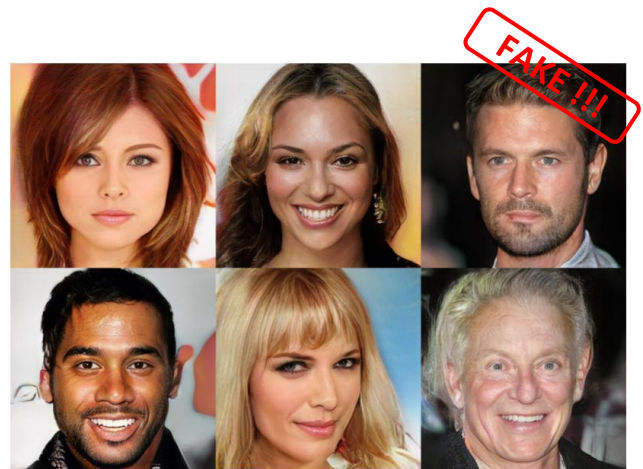


GANs - Applications



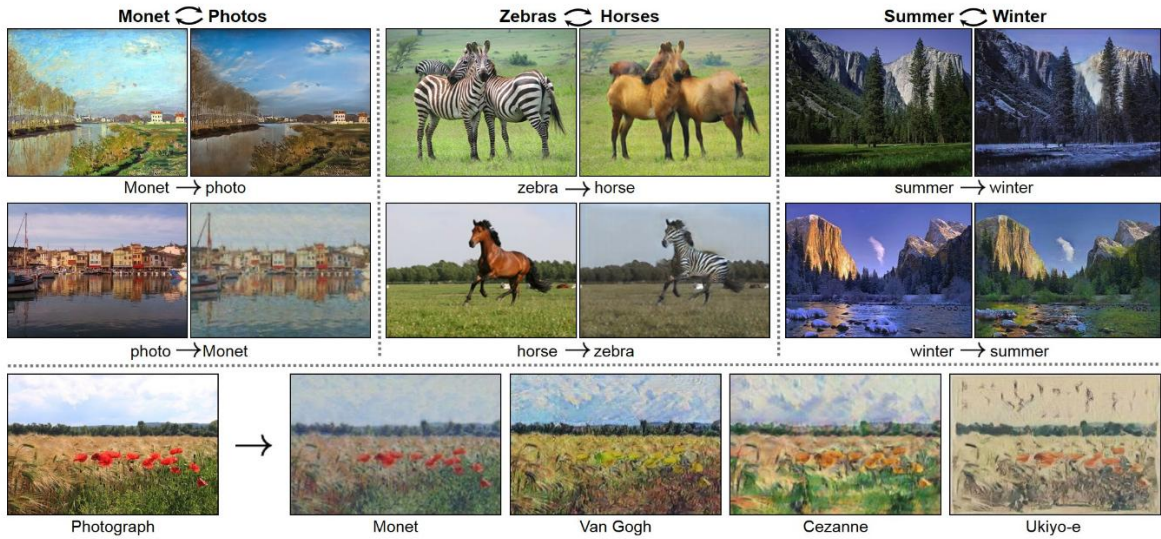
The GAN revolution

- Most modern architectures are based on the Deep Convolutional GAN (DCGAN), where the generator and discriminator are both CNNs.
- GANs revolutionised generative modelling by producing crisp, high-resolution images.
- Since GANs were introduced in 2014, there have been hundreds of papers introducing various architectures and training methods. GAN Zoo: <https://github.com/hindupuravinash/the-gan-zoo>



CycleGAN

- **Style transfer problem**: change the style of an image while preserving the content.

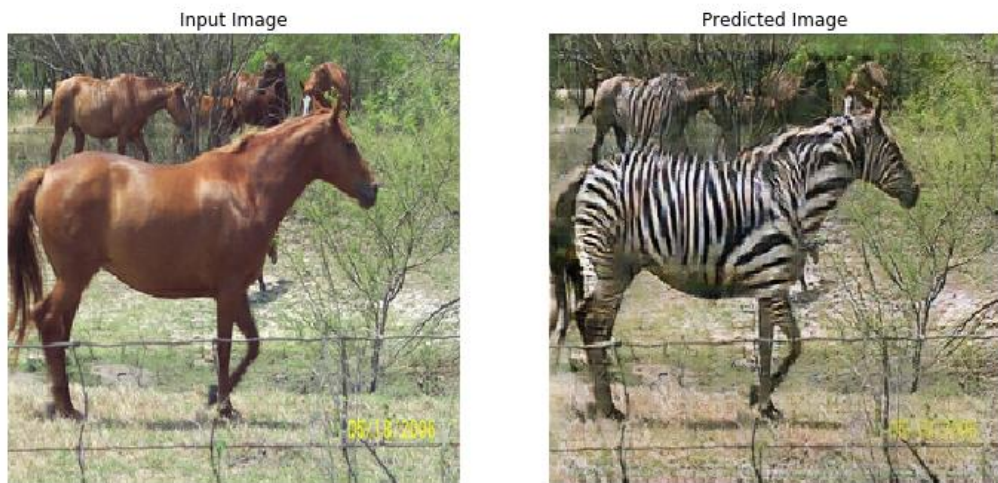


AI - Prof Ivan Olier

29

29

CycleGAN



AI - Prof Ivan Olier

30

30

CycleGAN



31

CycleGAN



32

CycleGAN

